

VHDL Implementation of NIST Ascon AEAD128 on a Nexys A7 FPGA

Kroix Jones

Electrical Engineering Undergraduate Researcher

Howard University

Research Advisor: Professor Wenjie Che

Abstract

This paper describes a VHDL-2008 implementation of the standardized NIST Ascon-AEAD128 algorithm for a Nexys A7 100T FPGA. The design uses one shared 320-bit permutation datapath and completes one Ascon round per clock cycle. A 128-bit streaming interface accepts associated data and payload bytes, including empty and partial blocks. Encryption returns ciphertext and a tag. Decryption returns tentative plaintext and releases it only after tag verification. The portable test flow passed all 1,089 official known-answer vectors in both directions with GHDL. Generated Verilog was also checked with Icarus Verilog and Verilator. Vivado scripts, a TextIO testbench, constraints, and a switch-and-LED demonstration are included. Vivado implementation measurements remain pending because Vivado is not available on the Mac used for this stage of the work.

I. Research objective

The goal of this work is to build an understandable FPGA implementation of authenticated encryption. The core must follow the final standard, handle normal streaming stalls, and make authentication failure visible to the surrounding system. The first version favors a clear control path over maximum throughput. It is meant to support later research on faster datapaths and hardware security countermeasures without changing the external transaction format.

II. Ascon-AEAD128 background

NIST SP 800-232 defines Ascon-AEAD128 with a 128-bit key, 128-bit nonce, 128-bit authentication tag, 128-bit data rate, and a 320-bit internal state [1]. The state is divided into five 64-bit words. Initialization and finalization use a 12-round permutation. Associated-data and full payload blocks use an 8-round permutation. Each round applies a constant to one state word, evaluates 64 five-input S-box slices in parallel, and then performs word-specific rotations and XOR operations.

The final NIST version was selected instead of legacy Ascon-128 or Ascon-128a. The final standard changed the initialization value and changed byte mapping to little endian [1]. For this core, byte lane 0 is bits 7 through 0 of the 128-bit port. Padding places the byte value 01 immediately after the last valid byte. Using the official mapping avoids a common result where a design is internally consistent but does not match current NIST vectors.

Initialize p12	Absorb AD p8	Domain separate	Process data p8	Finalize p12	Tag 128 bits
---------------------------------	-----------------	--------------------	--------------------	-----------------	-------------------------------

Fig. 1. Ascon-AEAD128 processing phases, adapted from NIST SP 800-232 [1].

III. Hardware architecture

The datapath contains five 64-bit state registers and one combinational round. A small round counter selects the required constants and returns the updated state after 8 or 12 clocks. Reusing one round reduces duplicated logic and keeps the relationship between the VHDL and the algorithm visible. The same engine serves initialization, associated data, payload processing, and finalization.

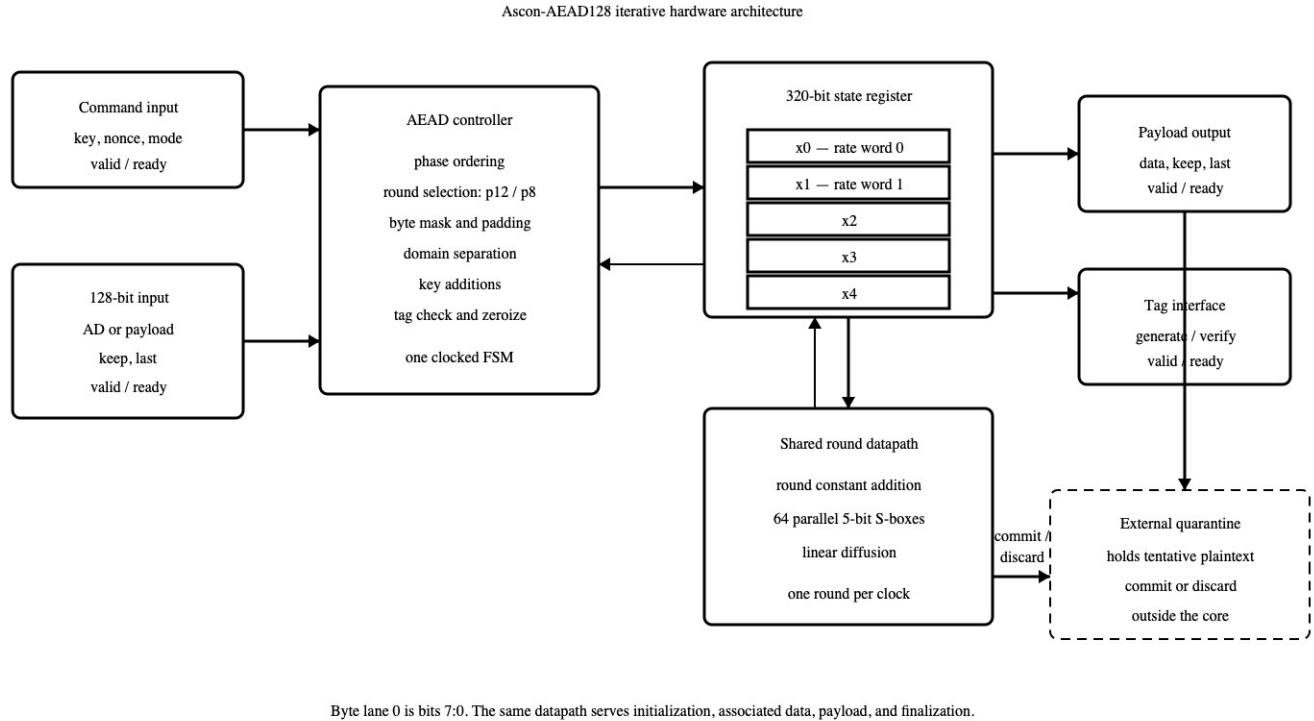


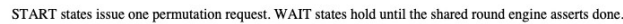
Fig. 2. Original block diagram of the implemented architecture.

The controller accepts a command containing the key, nonce, and direction. Associated data must finish before payload begins. Each stream beat includes 16 keep bits and a last flag. Keep bits must start at lane 0 and remain contiguous. An empty phase uses a terminal beat with no keep bits set. Any invalid ordering or mask produces a protocol error and sends all transaction storage through the same zeroization state used after a normal result.

IV. Authentication-safe decryption

A decryption core cannot know whether plaintext is authentic until it processes the final tag. Storing every possible message inside the cryptographic core would make its memory size depend on the application. This design sends tentative plaintext to an external quarantine buffer instead. A correct tag pulses commit. A wrong tag pulses discard and never pulses commit. The interface therefore makes the safe release rule explicit, but the system integrator must enforce it.

The finite-state machine was made for this streaming architecture. Start states issue a one-cycle request to the permutation engine. Wait states prevent another request until the current permutation ends. Output states hold data stable during backpressure. Tag states either wait for the encryption tag to be accepted or wait for a decryption tag to be supplied.



Verification began with an independent Python model. The model was compared with the official ascon-c known-answer file pinned to a recorded source revision [2]. Cocotb then drove the same vectors through the VHDL core. The full run covered all 1,089 combinations of associated-data and plaintext lengths from 0 through 32 bytes, with encryption followed by decryption. Separate tests stalled each output, reset an active transaction, sent invalid masks and phase ordering, checked fixed-seed messages up to 80 bytes, and changed the key, nonce, associated data, ciphertext, and tag.

Table I. Verification results collected before Vivado implementation.

VI. Vivado flow and FPGA demonstration

The repository contains Tcl scripts for Vivado 2023.2 and an XDC file for the Nexys A7 100T part xc7a100tcsg324-1. The constraint sets the board clock to 100 MHz. A pure VHDL XSim testbench writes `simulation_results.txt` for human review and `simulation_vectors.txt` for scripts. The same file-driven bench was checked with GHDL across all 1,089 vectors in both directions. XSim uses the identical TextIO input. The official-vector TextIO run measured 34 to 88 command-to-done cycles. For a 32-byte payload, the mean across all 33 associated-data lengths was 72.636 cycles, equal to 352.441 Mb/s at 100 MHz. These are cycle-based simulation results, not a measured maximum clock rate. XSim execution, synthesis, implementation, timing, and power reporting still require the Vivado machine. Resource use, maximum clock rate, and post-route throughput are intentionally not estimated in this paper.

The board wrapper stores four official vectors: empty input, a one-byte partial block, one full block, and a 17-byte two-block case. Switches choose the vector, encryption or decryption, payload or tag display, block number, and byte number. The center button starts a run and the upper button resets the design. LEDs display the selected result byte, busy and done state, authentication pass or fail, and a protocol error.

VII. Limitations and next work

This version is an unprotected cryptographic core. It does not prevent power or electromagnetic side-channel analysis, and it does not detect injected faults. Kandi et al. describe threshold and redundancy-based hardware options that could be studied after the baseline is stable [3]. A faster unrolled or ping-pong architecture is another option, but it would use more logic and make the first implementation harder to inspect [4]. The external quarantine buffer is also a required part of safe decryption, not an optional application feature.

VIII. Conclusion

The project now has a readable NIST Ascon-AEAD128 RTL core, an explicit streaming protocol, an authentication-safe decryption contract, official-vector tests, and a reproducible Vivado setup. Portable simulation gives evidence that the algorithm and boundary handling are correct. The next measured step is to run the prepared Vivado flow, record utilization and post-route timing, and test the four-vector demonstration on the Nexys A7 board.

References

- [1] M. S. Turan, K. A. McKay, D. Chang, J. Kang, and J. Kelsey, "Ascon-Based Lightweight Cryptography Standards for Constrained Devices," NIST SP 800-232, Aug. 2025, doi: 10.6028/NIST.SP.800-232.
- [2] Ascon Team, "ascon-c: Reference and optimized implementations of Ascon," commit 446347f21b209f3921c65ece70027c366cbe1693. [Online]. Available: <https://github.com/ascon/ascon-c>
- [3] A. Kandi et al., "Hardware Implementation of ASCON," NIST Lightweight Cryptography Workshop, June 2023.
- [4] K. Liu, L. Han, and Z. Quan, "A High Performance Ascon Architecture for Data Encryption and Authentication," in Proc. 5th Int. Conf. on Cryptography, Network Security and Communication Technology (CNSCT 2026), Haikou, China, Jan. 23–25, 2026, doi: 10.1145/3802927.3802941.
- [5] N. T. M. Huong, T. V. Nam, and P. V. Phat, "Application of the Ascon-AEAD128 Algorithm for IoT Data Security," 11th Int. Conf. on Applying New Technology in Green Buildings, 2026, doi: 10.1109/ATIGB70203.2026.11628316.
- [6] Digilent, "Nexys A7-100T Master XDC." [Online]. Available: <https://github.com/Digilent/digilent-xdc>